

Entrega 1: Modelado y Encapsulamiento

Explicar el concepto de encapsulamiento y su importancia en la programación orientada a objetos. Justificar la elección de atributos y métodos para modelar personajes de ciencia ficción. Analizar ventajas y desventajas de distintas representaciones de listas y pilas para gestionar inventarios y poderes.

El **encapsulamiento** es un principio de POO (Programación Orientada a Objetos) que consiste en proteger los datos internos de un objeto y permitir que solo se modifiquen mediante métodos controlados.

En este proyecto nos fue de gran ayuda para:

- Evitar niveles inválidos de habilidades/objetos
- Evitar inconsistencias internas
- Controlar cómo se agregan, modifican o eliminan elementos
- Mantener un modelo coherente del personaje (por ejemplo, cómo aumenta el poder total).

Dentro del proyecto en los **atributos del personaje** utilizamos:

- **nombre**: Es nombre del nombre del personaje, el mismo nos sirve para identificarlo y además para mostrar su información en pantalla. Por ejemplo: Goku, Vegeta, Androide 17.
- **especie**: Indicia de qué raza es nuestro personaje. En el universo de Dragon Ball esto es muy importante ya que, por ejemplo, un Saiyajin no tiene las mismas habilidades que un Namekiano. La especie hace que los personajes no se muestren todos iguales.
- **planeta**: Es el planeta de origen. Esto es clave para trabajar con universos conectados. Nos sirve para unir personajes al grafo de planetas que construimos más adelante en el trabajo. Por ejemplo: Piccolo [Planeta Namek], Gohan [La Tierra].
- **nivel_personaje**: (entre 1 y 10) representa el poder del personajes de manera general. Esto es importante porque multiplica el poder final del personaje. Por ejemplo, Goku es nivel 9 aproximadamente, mientras que Mr. Satán es nivel 3.
- **habilidades**: Es una lista donde guardamos las técnicas del personaje. Las habilidades tienen niveles del 1 al 5, como si fueran evolutivas. Algunos ejemplos son: *Kamehameha*, *Genki Dama*, *Mankankosappo*
- **inventario**: Es la lista de objetos que tiene el personaje, donde cada uno de ellos tiene su nivel y aporta poder. En Dragon Ball objetos claves son por ejemplo: las esferas del dragón o la semilla del Ermitaño.



Trabajo Final para la Cátedra: Estructuras de Datos en Python

Integrantes: Álvarez Rocío, Varela Ian, Gómez Morena.

Los métodos que utilizamos para modelar a los personajes de Dragon Ball fueron:

- **agregar_habilidad** (nombre, nivel): Sirve para añadir o mejorar habilidades., como cuando Goku domina mejor el Kamehameha.
- **quitar_habilidad** (nombre): Por si alguna habilidad deja de usarse o queremos modificar la lista.
- **agregar_objeto** (nombre, nivel): Agrega objetos al inventario, como si el personaje obtuviera algo nuevo durante la aventura.
- **quitar_objeto** (nombre): Elimina objetos si ya no sirven o si se gastaron (como una semilla usada).
- **poder_habilidades** (): Calcula la suma del poder proveniente de las técnicas, donde cada habilidad vale $200 \times$ nivel.
- **poder_objetos** (): Suma los puntos que le dan los objetos. Por ejemplo, una semilla nivel 1 da menos que una armadura nivel 5.
- **poder_total** (): Combina el poder de habilidades y objetos, multiplicándolo después por el nivel del personaje. Esto hace que un personaje con nivel bajo no sea fuerte aunque tenga muchas cosas. Por ejemplo, un Saiyajin con nivel 9 va a tener mucho más poder que Krillin nivel 4 aunque tengan técnicas parecidas.
- **mostrar** (): Muestra toda la información del personaje en pantalla.



Para gestionar inventarios y poderes nos inclinamos a elegir **Listas** ya que:

- ☒ Se puede recorrer toda la lista sin problemas.
- ☒ Podemos buscar habilidades como *Kamehameha* sin complicaciones.
- ☒ Podemos agregar o quitar elementos en cualquier orden.

Lo que sí, las listas pueden tener como desventajas:

- ☒ La búsqueda puede ser un poco más lenta.
- ☒ Podríamos repetir habilidades si no lo controlamos bien.

En tanto, las Pilas al funcionar como “LIFO” (último en entrar primero en salir) no las utilizamos ya que:

- ☒ No se puede acceder a elementos del medio.
- ☒ No sirven si queremos buscar, por ejemplo, “Kamehameha” dentro de la lista.
- ☒ No son prácticas para inventario ni habilidades.

Aunque algunas de sus ventajas hubieran sido que:

- ☒ No se puede acceder a elementos del medio.



Trabajo Final para la Cátedra: Estructuras de Datos en Python

Integrantes: Álvarez Rocío, Varela Ian, Gómez Morena.

- ☒ No sirven si queremos buscar, por ejemplo, “Kamehameha” dentro de la lista.
- ☒ No son prácticas para inventario ni habilidades

En conclusión, nos decidimos por listas porque:

- Se adapta mejor a cómo manejamos habilidades y objetos.
- Permite buscar y editar elementos fácilmente.
- Es similar a cómo uno imagina el “inventario” de un personaje de anime.
- Las pilas son interesantes, pero más útiles para transformaciones o estados temporales (como cuando Goku cambia de nivel en una batalla).

